

# Autonomous Kinematics Simulator & Navigation Controller: A Digital Twin for Non-Holonomic Vehicles

Hadi Ahmad

## Abstract

This document details the development of a 2D digital twin designed to simulate the kinematics of a non-holonomic differential drive robot. The project serves as a software-in-the-loop testing environment to validate closed-loop control algorithms prior to physical hardware deployment. By incrementally implementing proportional control, a state machine for mission planning, digital signal filtering, and full PID control, the system successfully navigates multi-waypoint missions while compensating for simulated real-world sensor noise and derivative control chatter.

## 1 Introduction & Project Rationale

In industrial automation and mobile robotics, deploying untested control algorithms directly to physical hardware risks damaging expensive equipment due to logic errors or unstable tuning. A standard engineering practice is to first develop a "digital twin", a mathematical simulation of the physical plant.

This project establishes a simulated environment for a two-wheeled differential drive robot. The primary objective is to develop a robust navigation controller capable of guiding the vehicle through a sequence of coordinates (a mission profile) while actively mitigating the effects of noisy sensor data, a common reality in embedded systems relying on inexpensive IMUs or wheel encoders.

## 2 System Development Methodology

### 2.1 Phase 1: Forward Kinematics Modeling

The foundation of the digital twin relies on accurately translating individual wheel velocities into 2D spatial movement. For a robot with wheel radius  $r$  and wheelbase  $d$ , the linear velocity  $v$  and angular velocity  $\omega$  are calculated from the right ( $\omega_R$ ) and left ( $\omega_L$ ) wheel speeds:

$$v = \frac{r}{2}(\omega_R + \omega_L) \quad (1)$$

$$\omega = \frac{r}{d}(\omega_R - \omega_L) \quad (2)$$

The system state  $(x, y, \theta)$  is updated using numerical integration over time step  $\Delta t$ :

$$x_{t+1} = x_t + v \cos(\theta_t) \Delta t \quad (3)$$

$$y_{t+1} = y_t + v \sin(\theta_t) \Delta t \quad (4)$$

$$\theta_{t+1} = \theta_t + \omega \Delta t \quad (5)$$

## 2.2 Phase 2: Closed-Loop Proportional Control & State Machine

To enable autonomous navigation, a proportional (P) controller was implemented to minimize both distance error ( $e_d$ ) and heading error ( $e_\theta$ ) relative to a target waypoint.

A state machine was introduced to handle multi-waypoint mission profiles. When  $e_d$  falls below a defined tolerance threshold (0.05 meters), the state machine advances the target index. To prevent the non-holonomic vehicle from executing wide, unoptimized arcs during sharp turns, a "Pivot-then-Drive" behavioral logic was implemented, forcing forward velocity to zero if the heading error exceeded  $15^\circ$ .

## 2.3 Phase 3: Real-World Simulation (Sensor Noise & DSP)

A control system is only as reliable as its sensor data. To simulate physical hardware constraints, Gaussian noise ( $\sigma = 0.03\text{m}$ ) was injected into the ground-truth ( $x, y$ ) coordinates.

Feeding raw, noisy data directly into the navigation controller results in erratic behavior. To resolve this, an Exponential Moving Average (EMA) low-pass filter was applied to the incoming data stream, blending new measurements with historical estimates to produce a smooth, actionable state belief.

$$\text{est\_x} = (\alpha \times \text{measured\_x}) + ((1 - \alpha) \times \text{est\_x}) \quad (6)$$

where  $\alpha = 0.2$ , dictating an 80% reliance on historical data.

## 2.4 Phase 4: Full PID Implementation and Chatter Mitigation

To optimize the dynamic response and minimize steady-state errors, the heading controller was upgraded to a full Proportional-Integral-Derivative (PID) architecture.

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (7)$$

During initial testing, the Derivative term exhibited severe "control chatter." Even with the positional EMA filter, micro-fluctuations in the calculated error slope caused the D-term to oscillate violently, resulting in a jittery physical trajectory.

This was resolved by implementing a secondary, highly aggressive low-pass filter exclusively for the derivative calculation:

```
1 # Calculate raw rate of change
2 delta_error = heading_error - prev_heading_error
3 delta_error = math.atan2(math.sin(delta_error), math.cos(delta_error))
4 raw_derivative = delta_error / dt
5
6 # Apply heavy secondary filter to the derivative term (alpha = 0.1)
7 filtered_derivative = (0.1 * raw_derivative) + (0.9 * prev_derivative)
8 prev_derivative = filtered_derivative
9
10 D_out = Kd_angular * filtered_derivative
```

Listing 1: Derivative Chatter Mitigation via Secondary Low-Pass Filter

## 3 Conclusion

This project successfully bridged theoretical vehicle kinematics with applied digital signal processing and control theory. By mathematically validating the PID tuning and filtering algorithms within a digital twin, the resulting logic is proven stable and ready for direct porting into a C++ environment for physical embedded deployment.

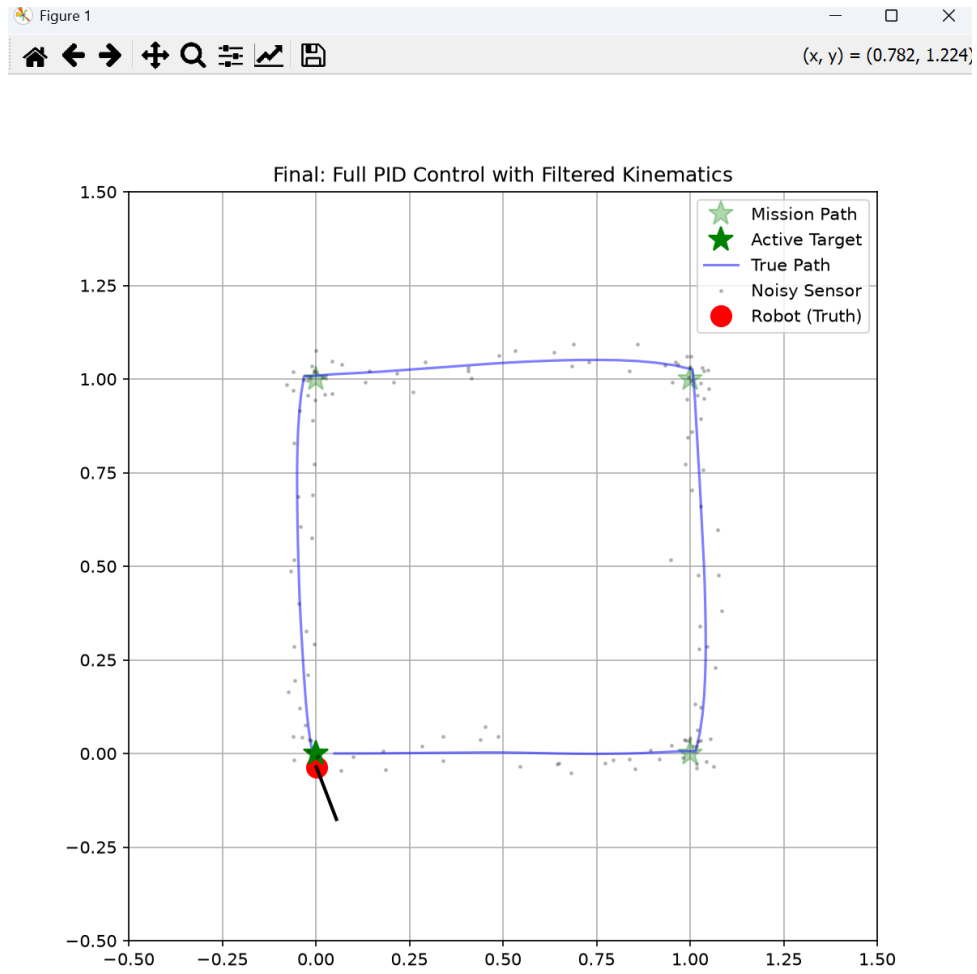


Figure 1: Final Trajectory: Full PID Control actively tracking through simulated Gaussian sensor noise.